PROGRAMA EDUCATIVO EN:

Tecnologías de la Información en Desarrollo y Gestión de Software

DESARROLLO WEB PROFESIONAL

NOMBRE DEL ESTUDIANTE:

Héctor Ulises Cacho González

Dana Lizbeth Castañeda Sánchez

Oscar Yael Hernández Rodríguez

Concepción Guadalupe Paniagua González

Ciro Julián Cervantes Zamora

S - 7. COMUNICACIÓN ASÍNCRONA ACCESIBLE

8VO CUATRIMESTRE “A”

Manual de Comunicación Asíncrona Accesible

El presente documento constituye la normativa técnica oficial para el ecosistema RAPITI. Define de manera exhaustiva los lineamientos de arquitectura, patrones de diseño, protocolos de comunicación y estándares de calidad. Su cumplimiento es obligatorio para todos los roles (Frontend, Backend, DevOps, QA) con el fin de asegurar la interoperabilidad, escalabilidad y mantenibilidad del producto.

Consumo de APIs y Comunicación Asíncrona

Patrón de Servicio Robusto

Toda interacción con servicios externos debe centralizarse en una capa de servicios independiente para desacoplar la lógica de negocio de la interfaz.

Ejemplo de Consumo Funcional:

```
/*
 * Servicio estandarizado para la obtención de recursos.
 * @param {string} entity - Entidad a consultar (ej: 'products').
 * @param {AbortSignal} signal - Controlador para evitar race conditions.
 */
export const apiService = async (entity, signal) => {
  const URL = `${import.meta.env.VITE_API_URL}/${entity}`;

  try {
    const response = await fetch(URL, {
      headers: { 'Content-Type': 'application/json' },
      signal
    });

    if (!response.ok) {
      const errorData = await response.json();
      throw { status: response.status, ...errorData };
    }

    return await response.json();
  } catch (error) {
    if (error.name === 'AbortError') return null;
    throw error;
  }
};
```

Gestión de Concurrencia

Es obligatorio el uso de AbortController en los hooks de efecto para limpiar peticiones pendientes al desmontar componentes o cambiar de ruta, garantizando que el estado del frontend sea consistente con la última acción del usuario.

Estados de Carga Accesibles (UX/A11Y)

Loaders y Skeletons

La aplicación no debe presentar pantallas vacías ni estados de bloqueo sin feedback visual.

- Skeletons: Se deben utilizar para cargas de listas o detalles de productos, manteniendo el *layout* original para evitar el CLS (*Cumulative Layout Shift*).
- Atributos ARIA: Los elementos de carga deben incluir aria-busy="true" y aria-live="polite" para que los lectores de pantalla informen que el contenido se está actualizando.
- App Loader: Un loader global debe cubrir la inicialización de la app (auth check) antes del primer renderizado.

Gestión Global de Errores

Protocolo de Errores Estructurados

El backend debe retornar siempre un objeto de error con la siguiente jerarquía:

1. success: false.
2. error.code: Identificador único (ej: ERR_CONNECTION_TIMEOUT).
3. error.message: Mensaje descriptivo para el usuario.
4. error.details: Datos adicionales para depuración o validación de formularios.

Feedback Correctivo

El frontend debe implementar un "Error Boundary" o un manejador de excepciones que ofrezca al usuario una salida clara:

- Botón de "Reintentar".
- Mensajes de error humanos (evitar códigos técnicos crudos).
- Persistencia de datos locales en caso de fallos de red (Offline support básico).

Estrategia de Despliegue Parcial

CI/CD y Deploy Activo

Se utiliza GitHub Actions para automatizar el ciclo de vida del software mediante despliegues parciales controlados.

- Build Multistage: Uso de Docker para separar la compilación del entorno de ejecución (Nginx).
- Feature Flags: Se recomienda el uso de variables de entorno para habilitar o deshabilitar módulos en despliegues parciales (ej: VITE_ENABLE_NEW_SEARCH=false).
- Smoke Tests: Cada deploy parcial debe validar que los servicios críticos (Login, Listado) estén operativos antes de completar la rotación de contenedores.

Entregables Obligatorios

Entregable	Criterio de Aceptación
Consumo de API funcional	Peticiones documentadas, asíncronas y con manejo de estados idle/loading/success/error.
Estados de carga accesibles	Uso de Skeletons con roles ARIA y cumplimiento de contraste visual.
Manejo de errores documentado	Diccionario de errores en backend y componentes de alerta consistentes en frontend.
Deploy parcial activo	Pipeline de integración continua con validación de tests y build en Docker.

Este manual garantiza que RAPITI no sea solo una aplicación funcional, sino una plataforma robusta, profesional y accesible. La adherencia estricta a los protocolos de consumo, error y despliegue facilita el crecimiento del producto y la eficiencia del equipo.

Referencias Bibliográficas

Berners-Lee, T., Fielding, R., & Masinter, L. (2005). *Uniform Resource Identifier (URI): Generic Syntax*. RFC Editor. https://doi.org/10.17487/RFC3986

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Disertación doctoral, University of California, Irvine]. https://www.google.com/search?q=https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Google.(s.f.).*Web Vitals: Cumulative Layout Shift (CLS)*.web.dev.

https://web.dev/articles/cls

Mozilla Social. (2023). *MDN Web Docs: Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.

https://www.nngroup.com/books/usability-engineering/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/www-project-top-ten/

React. (s.f.). *Error Boundaries*. React Documentation.

https://www.google.com/search?q=https://react.dev/reference/react/Component%23catching-rendering-errors-with-an-error-boundary

Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media. https://www.oreilly.com/library/view/restful-web-services/9780596529260/

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*.

https://www.w3.org/TR/WCAG22/